

Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчёт к практическому заданию №1
по дисциплине «Системное программное обеспечение»

Студент: Горинов Даниил Андреевич
Группа: Р4116

Санкт-Петербург
2025/2026 учебный год

1 Цели

Использовать средство синтаксического анализа для разбора языка по варианту, построить синтаксическое дерево с узлами всех конструкций языка и обеспечить вывод дерева в графическом формате.

2 Задачи

Порядок выполнения:

1. Изучить выбранное средство синтаксического анализа
 - (a) Средство должно поддерживать программный интерфейс, совместимый с языком Си
 - (b) Средство должно параметризоваться спецификацией, описывающей синтаксическую структуру разбираемого языка
 - (c) Средство может функционировать посредством кодогенерации и/или подключения необходимых для его работы дополнительных библиотек
 - (d) Средство может быть реализовано с нуля, в этом случае оно должно использовать обобщённый алгоритм, управляемый спецификацией
2. Изучить синтаксис разбираемого по варианту языка и записать спецификацию, включающую следующие конструкции:
 - (a) Подпрограммы со списком аргументов и возвращаемым значением
 - (b) Операции контроля потока управления — простые ветвления if-else и циклы или аналоги
 - (c) В зависимости от варианта — определения переменных
 - (d) Целочисленные, строковые и односимвольные литералы
 - (e) Выражения численной, битовой и логической арифметики
 - (f) Выражения над одномерными массивами
 - (g) Выражения вызова функции
3. Реализовать модуль, использующий средство синтаксического анализа для разбора языка по варианту
 - (a) Программный интерфейс модуля должен принимать строку с текстом и возвращать структуру, описывающую соответствующее дерево разбора и коллекцию сообщений об ошибке
 - (b) Результат работы модуля — дерево разбора — должен содержать иерархическое представление для всех синтаксических конструкций, включая выражения, логически представляющие собой иерархически организованные данные, даже если на уровне средства синтаксического анализа для их разбора было использовано линейное представление
4. Реализовать тестовую программу для демонстрации работоспособности созданного модуля

- (a) Через аргументы командной строки программа должна принимать имя входного файла для чтения и анализа, имя выходного файла записи для дерева, описывающего синтаксическую структуру разобранного текста
 - (b) Сообщения об ошибке должны выводиться тестовой программой (не модулем, отвечающим за анализ) в стандартный поток вывода ошибок
5. Результаты тестирования представить в виде отчета, в который включить:
- (a) В части 3 привести описание структур данных, представляющих результат разбора текста
 - (b) В части 4 описать, какая дополнительная обработка потребовалась для результата разбора, предоставляемого средством синтаксического анализа, чтобы сформировать результат работы созданного модуля
 - (c) В части 5 привести примеры исходных анализируемых текстов для всех синтаксических конструкций разбираемого языка и соответствующие результаты разбора

3 Описание работы

Синтаксическая модель варианта

Вариант задаёт статическую систему типов, где типы указываются в сигнатурах функций и параметров. Поддержаны идентификаторы, строковые и символьные литералы, а также двоичные/шестнадцатеричные и десятичные целые. Лексические правила соответствуют заданным регулярным выражениям:

```

identifier: [a-zA-Z_] [a-zA-Z_0-9]*
str: "[^"\\]*(?:\\. [^"\\]*)*"
char: '[^']*'
hex: 0[xX] [0-9A-Fa-f]+
bits: 0[bB] [01]+
dec: [0-9]+
bool: true|false
list<item>: (item (',' item)*)?

```

Специфическая часть грамматики реализует `typeRef`, сигнатуры функций, операторы `if/else`, `while/until`, `repeat`, `break`, блоки, выражения, вызовы и индексацию массивов.

Внешнее описание программы и интерфейс

CLI-утилита принимает список файлов и директорию вывода:

```
./app <input_files...> <output_dir>
```

Для каждого входного файла строится DGML-файл вида `ast.<source>.dgml`. Диагностика лексера/парсера выводится в стандартный поток ошибок.

Структуры данных результата разбора

AST реализован структурой `AST_Node` с типом узла, данными литералов и массивом дочерних узлов для составных конструкций. Для последовательностей используется `AST_LIST`.

```
typedef struct AST {
    NodeType type;
    char* op;
    union {
        struct { uint32_t num_val; char* num_str; };
        char* type_name;
        char* string_value;
        char char_value;
        char* id;
        struct { int child_count; struct AST** children; } compound;
    };
} AST_Node;
```

Модуль возвращает `AST_Wrap` с именем файла, деревом и коллекцией сообщений (предупреждений).

Дополнительная обработка результата разбора

Парсер Bison строит дерево на уровне правил. Для обеспечения строгой иерархичности дополнительно используются узлы-обертки: `AST_LIST` — для списков аргументов и операторов, `AST_CALL` — для вызовов, `AST_SLICE` и `AST_RANGE` — для индексации и срезов. Это устраняет линейное представление и фиксирует структуру выражений, что требуется заданием.

4 Аспекты реализации

Лексический анализ и координаты

Flex-лексер заполняет `yylloc`, обеспечивая точные координаты токенов. Ключевые токены (`bool/int/uint/long/char/string`, `def/if/then/else/while/until`, операторы сравнения и логики) сопоставлены с терминалами грамматики.

Построение AST

В грамматике выражения имеют стандартный приоритет: унарные операции, мультипликативные, аддитивные, сравнения, равенства, логические AND/OR. Семантические действия создают узлы `AST_ADD`, `AST_MUL`, `AST_EQ` и т.д., сохраняя оператор в поле `op` для последующего использования. Ветвления создают `AST_IF` с опциональным `AST_ELSE`. Циклы отражаются узлами `AST_WHILE`, `AST_UNTIL`, `AST_REPEAT_WHILE` и `AST_REPEAT_UNTIL`.

Определения переменных по варианту

В языке варианта отдельной конструкции объявления переменных нет; переменные вводятся при первом присваивании. Это соответствует требованию «в зависимости

от варианта — определения переменных».

Экспорт в DGML

Экспортёр обходит дерево и формирует узлы и связи. Для литералов и идентификаторов выводятся подписи с их значениями, что позволяет визуально проверять дерево.

5 Результаты

- Реализованы лексер и парсер Flex/Bison, поддерживающие все конструкции варианта.
- Создано AST с иерархическим представлением выражений и управляющих конструкций.
- Реализована CLI-программа и экспорт AST в DGML.
- Тест `tests/input.txt` покрывает: функции, `if/else`, `while/until`, `repeat`, `break`, литералы, вызовы, индексацию/срезы.
- Пример артефакта: `results/ast.input.txt.dgml`.

6 Выводы

Сформирован модуль синтаксического анализа, который по тексту программы строит полное дерево разбора и экспортирует его в графическом формате. Реализация соответствует требованиям задания, поддерживает статически типизированные сигнатуры функций, управляющие конструкции и операции над массивами, а также предоставляет основу для построения CFG на следующем этапе.